Figure 11.5 Swagger UI groups the endpoints in your application based on the `Tag` metadata attached to them. The UI uses the metadata added by calling `Produces()` to document the expected return types and status codes for each endpoint.

If adding all this extra metadata feels like a bit of a chore, don't worry. Adding the extra OpenAPI metadata is optional, necessary only if you plan to expose your OpenAPI document for others to consume. If all you want is an easy way to test your minimal APIs, you can go a long way without many of these extra method calls.

TIP Remember that you can also use route groups (described in chapter 5) to apply metadata to multiple APIs at the same time.

One of the strongest arguments for making your OpenAPI descriptions as rich as possible is that it makes the tooling around your API easier to use. Swagger UI is one example. But an arguably even more useful tool lets you automatically generate C# clients for interacting with your APIs.

11.4 Generating strongly typed clients with NSwag

In this section you'll learn how to use your OpenAPI description to automatically generate a client class that you can use to call your API from another C# project. You'll create a console application, use a .NET tool to generate a C# client for interacting with your API, and finally customize the generated types. The generated code includes automatic serialization and deserialization of request types, and makes interacting with your API from another C# project much easier than the alternative method of crafting HTTP requests manually.

NOTE Generating a strongly typed client is optional. It makes it easier to consume your APIs from C#, but if you don't need this functionality, you can still